

Alma Mater Studiorum Università di Bologna
Archivio istituzionale della ricerca

ScaFi-Blocks: A Visual Aggregate Programming Environment for Low-Code Swarm Design

This is the final peer-reviewed author's accepted manuscript (postprint) of the following publication:

Published Version:

Aguzzi, G., Casadei, R., Cerioni, M., Viroli, M. (2024). ScaFi-Blocks: A Visual Aggregate Programming Environment for Low-Code Swarm Design. Springer Science and Business Media Deutschland GmbH [10.1007/978-3-031-62697-5_14].

Availability:

This version is available at: <https://hdl.handle.net/11585/999395> since: 2025-03-21

Published:

DOI: http://doi.org/10.1007/978-3-031-62697-5_14

Terms of use:

Some rights reserved. The terms and conditions for the reuse of this version of the manuscript are specified in the publishing policy. For all terms of use and more information see the publisher's website.

This item was downloaded from IRIS Università di Bologna (<https://cris.unibo.it/>).
When citing, please refer to the published version.

(Article begins on next page)



ScaFi-Blocks: A Visual Aggregate Programming Environment for Low-Code Swarm Design

Gianluca Aguzzi^[0000–0002–1553–4561], Roberto Casadei^[0000–0001–9149–949X],
Matteo Cerioni, and Mirko Viroli^[0000–0003–2702–5702]

Alma Mater Studiorum – Università di Bologna
{gianluca.aguzzi, roby.casadei, mirko.viroli}@unibo.it
matteo.cerioni2@studio.unibo.it

Abstract. Swarm programming is focused on the design and implementation of algorithms for large-scale systems, such as fleets of robots, ensembles of IoT devices, and sensor networks. Writing algorithms for these systems requires skills and familiarity with programming languages, which can be a barrier for non-expert users. Even if visual programming environments have been proposed for swarm systems, they are often limited to specific platforms or tasks, and do not provide a high-level programming model that can be used to design algorithms for a wide range of swarm systems. Therefore, in this paper, we propose a low-code swarm programming environment, called ScaFi-Blocks, which allows users to design and implement swarm algorithms visually. ScaFi-Blocks is based on the ScaFi aggregate computing framework, which provides a high-level programming model for the design of distributed algorithms. Aggregate computing is based on the concept of field-based coordination, and it allows users to design algorithms by composing simple building blocks, which motivates the design of the proposed artefact. The environment is designed to be user-friendly and to support the design of a wide range of collective applications. In this paper, we present the architecture of ScaFi-Blocks, discuss its features and capabilities, and provide a preliminary evaluation of the programming environment based on a case study featuring articulated swarm behaviour.

Keywords: field-based coordination · low-code · visual programming · aggregate computing · swarm robotics · macroprogramming

1 Introduction

Swarm programming is an emergent approach for devising collective algorithms within extensive, decentralized systems like swarms of robots, IoT device ensembles, and sensor networks. The intrinsic complexity and dynamic nature of such systems present challenges in articulating desired collective behaviours succinctly and effectively. Traditional device-centric programming paradigms fall short in addressing the needs of large-scale system design, amplifying the cognitive load and widening the gap between the programmers’ intentions and the achievable

solutions [38]. To mitigate these issues, several solutions that shift the focus from individual devices to collective behaviours have been proposed. Among these, macroprogramming [17] and automatic design [26] stand out as prominent directions. Macroprogramming models, particularly *aggregate computing* [8], treat collectives as first-class entities simplifying the devising of collective behaviours. Conversely, automatic design employs techniques like evolutionary robotics [27] and multi-agent reinforcement learning [28] to navigate the vast design space of swarm algorithms, which might be infeasible to explore manually. However, both avenues demand substantial expertise and/or a deep programming knowledge, posing barriers to entry for non-specialists. Furthermore, designing algorithms for swarm systems remains a daunting task due to the inherent complexity, high cognitive load, and susceptibility to errors. In related fields, low-code solutions (i.e., environments that allow users to design applications with minimal coding), especially those employing *visual programming* [39,45] environments, have been proposed to lower these barriers and reduce cognitive load, facilitating the design of applications in a more intuitive and accessible manner.

For such reasons, this paper introduces *ScaFi-Blocks*, a low-code swarm programming environment that leverages visual programming to *democratize* the development of swarm algorithms. Building on the ScaFi [20] aggregate computing framework and Blockly visual programming library ¹, ScaFi-Blocks enables users to visually compose algorithms using simple, yet powerful, building blocks. This approach not only lowers the entry barrier for utilizing the ScaFi framework but also supports the development of a broad spectrum of collective applications with reduced cognitive effort and error proneness thanks to the compositional nature of aggregate computing. In the following, we outline the architecture of ScaFi-Blocks, discuss its distinctive features and capabilities, and present a preliminary evaluation through a case study some common field-based coordination algorithms. This contribution fills a notable gap in the literature by providing a visual low-code programming environment specifically designed for swarm systems, thereby facilitating broader access and innovation in the field of swarm programming.

2 Background

2.1 Visual Programming and Low-Code Engineering

Visual programming refers to programming approaches based on the use and arrangement of graphical elements [48,33,41,21], in contrast with traditional programming approaches based on textual programs. Visual programming approaches are mainly motivated by (i) pedagogical reasons, namely to help beginners learn the basics of programming (cf. Scratch [5]), and (ii) methodological and practical reasons, beyond education, primarily to support end-users and domain experts to create and tailor applications [33]. Indeed, visual programming can help to (i) reduce the entry barrier to tools, through simplified notation,

¹ <https://developers.google.com/blockly>

(ii) reduce the cognitive load, by simplifying the presentation of relevant information, (iii) reduce the likelihood of errors, through suitable constraints to the arrangements of visual elements, and (iv) foster the collaboration between domain experts and programmers, through a notation that the former group of users can more easily understand.

A related concept and keyword to visual programming is *low-code*, namely the idea and practice of enabling the development of applications with limited source code writing [44]. Therefore, visual programming is a means for achieving low-code or even *no-code* [47]. As in visual programming, the focus is both on empowerment of non-programmers and productivity in terms e.g. of proper collaboration with domain experts [9]. Typical approaches followed by visual and low-code platforms or environments [9] to achieve their goals include supporting data management, workflow management, and providing integrated development environments (IDEs).

2.2 Swarm Engineering

Engineering swarms is a significant research challenge [12] that is related to finding effective ways to express a global behaviour able to solve problems or provide services in a possibly complex environment. Swarm design methods can be classified into two main classes of approaches [12,16]: (i) *automatic* design methods, where algorithms are used to learn or synthesise behaviours (e.g. as driven by signals like rewards or fitness metrics—cf. multi-agent reinforcement learning [14] and evolutionary robotics [49]); and (ii) *manual* design methods, where algorithms are implemented by the designers using a classical iterative, incremental, trial-and-error process, often supported by specific *domain-specific languages (DSLs)* [43,3]. In this work, we focus on the latter kind of approaches, namely on what can be referred to as *swarm programming* [43,3].

Swarm programming can be considered under the umbrella of *macro-programming* [17], the abstract paradigm of expressing the macro-level or collective behaviour in systems featuring multiple interacting components. In this context, various kinds of concrete approaches and corresponding languages have been proposed, featuring different kinds of macro-level abstractions capturing, e.g., space-time structures (cf. SpatialViews [40] and field-based coordination [36,51]), collective communication (cf. attribute-based communication [1] and SCEL [23]), dynamic groups of devices (cf. *ensembles* [1,23]), and distributed values (e.g., *computational fields* [36,51]).

Specifically to swarm programming, a further distinction is sometimes made between (i) *task orchestration languages*, where task descriptions are used by centralised entities to give instructions to the swarm members (such as in TeCoLa [32], Dolphin [35], Maple-Swarm [31], PARoS [24], Resh [15]), and (ii) *decentralised swarm programming languages*, which instead focus on the definition of the control program of the swarm members, either by a traditional, node-level (e.g., multi-agent oriented programming [10]) or a more macroscopic perspective (e.g., MacroSwarm, Buzz).

Additionally, recurrent types of collective and self-organising behaviour have been described under the form of *patterns* [25,42,50,12], then sometimes also provided as corresponding DSL constructs for swarms as in *MacroSwarm* [3] or *Buzz* [43]. The ability of designing swarm behaviour through *compositions* of patterns, or *blocks* of collective behaviour, is a powerful feature. This is often provided by task-orchestration languages, which however generally provide centralised implementations, whereas it is most interesting when supported by decentralised swarm programming languages.

2.3 Aggregate Computing

Aggregate computing [51] is an approach for programming collective adaptive and self-organising systems such as swarms.

System model Aggregate computing assumes a system of computing *devices* able to exchange messages with *neighbours*. A device is assumed to be equipped with *sensors* and *actuators*, their only interface with the environment. Also, each device has a *buffer* for storing the most recent message of its neighbours.

Execution model Aggregate computing generally adopts an execution strategy that resembles the local, decentralised, repetition-based dynamics of self-organisation [11,18]. Each device computes at *asynchronous sense-compute-interact rounds*, namely a round consists of three main steps:

1. *sense*: the device gathers all the relevant information from its local context, which includes the most recent and *not-expired* messages from neighbours and the environment state (as sampled by sensors);
2. *compute*: the device evaluates the aggregate program (see next) against its local context, resulting in an *output* and a coordination message called an *export*;
3. *interact*: the device interacts with its environment, by *broadcasting* the export to all its neighbours (a step needed to support collective coordination) and performing *actuations* as dictated by the output of the program.

Notice that the model does not fix a number of details such as the concrete neighbouring relationship, scheduling strategy for rounds, message expiration, communication technology etc. Indeed, such details can be fine-tuned specifically to the application scenario at hand and even dynamically at the middleware level to achieve non-functional tradeoffs. Many of such details are not required for understanding the paradigm, and specifically for this paper, and are actually addressed by research such as [19,18].

Programming model The *aggregate program* is what defines the collective behaviour and is evaluated at the compute step by all the devices. An aggregate program is expressed as a composition of functions of computational fields.

A *computational field* [36,51] is a map from devices to values, and the main abstraction of aggregate computing, which is formalised in terms of calculi of computational fields or variants thereof [51,6]. The details of field calculi and their constructs are beyond the scope of this paper, which instead is about low-code programming. What is more important to understand, is that computational fields provide a denotation of collective inputs, collective outputs, and collective results (of expression evaluation). For instance, having all the devices in the system read their temperature sensor yields a field of temperatures; also, by relating this field to the field of their positions in space, the system provides a map of the temperatures in an environment. Also, to control a swarm, we can use fields of actuations; for instance, collective movement can be represented as a field of velocity vectors. Similarly, the evaluation of a local Boolean expression by all the devices in the system results in a Boolean field mapping each device to a truth value to denote, e.g., what are the sinks of a distributed collection process, or the leaders of the swarm. Another use of fields is for supporting *interaction* in the system: a device may process information from its neighbours in terms of a so-called *neighbouring field*, which maps each neighbour to their corresponding value for a given expression. So, for instance, the neighbouring field of temperature readings allows a device to access the temperatures in its surroundings.

A collective behaviour can be expressed as a function from fields to fields, whose body is an expression involving fields. Internally, such function may also use neighbouring fields to support communication; hence, a function becomes a *block* or *reusable component* that embeds all is needed to realise some collective-level behaviour. Also, crucially, functional composition can be used to express more complex collective behaviours in terms of simpler ones.

Example: gradient The gradient is one of the most fundamental self-organisation building blocks. It is the computation of the field of numbers denoting the minimum distances from source nodes. Such a computation can be encapsulated as a function from the Boolean field of sources to the numeric field of distances. Invoking a gradient, then, requires each device to evaluate it in each one of its rounds, and to share with neighbours the information dictated by such block. A simple implementation of the gradient is as follows: the source outputs a null distance, whereas the other devices output the minimum value over the neighbouring field of gradients augmented with the distances to those neighbours. The repeated evaluation of such logic results in a self-stabilising gradient that will eventually converge to the stable (i.e. non-changing) correct field once the environment gets stable.

Toolchain An aggregate computing toolchain includes languages, middlewares (including simulators), and development environments. A prominent toolchain implementation is ScaFi [20,7], which includes a Scala-internal DSL for writing aggregate programs, an internal simulator, and other tools. Among the tools, we mention ScaFi-Web [2], which is a web-based development environment that

streamlines the experimentation with the language with an accessible frontend and a set of pre-defined programs and scenarios. To date, there exist no visual or low-code implementations for aggregate computing. This work provides ScaFi-Blocks, by extending ScaFi-Web with a support for block-based visual programming.

3 ScaFi-Blocks

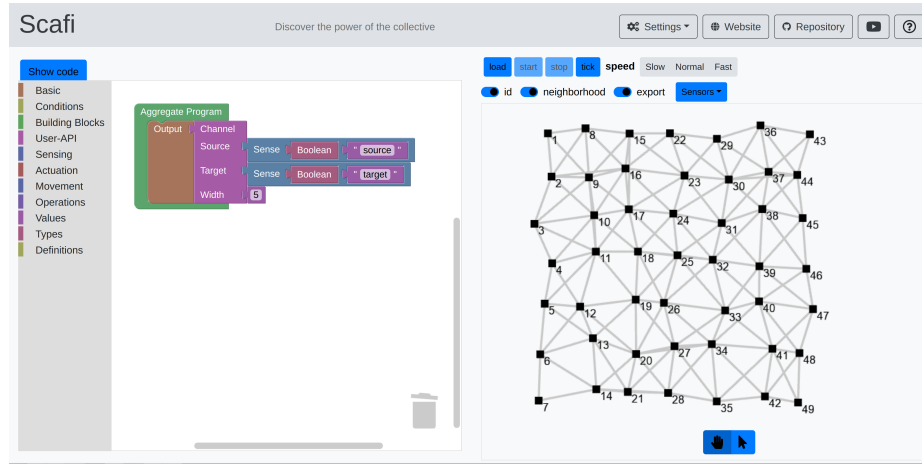


Fig. 1: The ScaFi-Blocks visual programming environment

ScaFi-Blocks (Figure 1) is a visual programming environment that leverages Blockly, a web-based visual programming library developed by Google. The page is divided into three main areas:

- The toolbox, which contains the building blocks that can be used to design algorithms (the left part in the picture).
- The workspace, where the user can design the algorithm by dragging and dropping the building blocks from the toolbox (the central part).
- The simulation area, where the user can execute the algorithm and visualize the behaviour of the swarm (the right part).

ScaFi-Blocks is seamlessly integrated with ScaFi-Web, a robust web-based platform designed for the creation and execution of aggregate computing algorithms. This integration fosters an intuitive and accessible means for users, particularly those without extensive expertise in aggregate computing, to engage with and understand the core principles of this computational paradigm. Indeed, ScaFi-Blocks exposes some of the most common building blocks of the ScaFi framework, allowing users to compose algorithms visually, and creating

some of the most idiomatic field-based coordination algorithms and then execute them on the ScaFi-Web platform. Simplifying the design of algorithms in this manner not only reduces the cognitive load and error proneness but also fosters collaboration, allowing non-expert users to contribute to the design of algorithms.

3.1 Features

Core Blocks: ScaFi-Blocks introduces a suite of core building blocks designed to encapsulate the essential principles of the ScaFi framework, making aggregate computing concepts accessible and intuitive. These blocks include:

- **Conditions:** blocks for performing conditional operations, enabling local decisions with **mux** or spatial decisions with **branch** construct, facilitating complex logic based on both local state and neighbourhood information.
- **Basic AC blocks:** a curated selection of ScaFi primitives focused on spatial operations, such as gradient-based computation, to introduce users to aggregate computing’s spatial reasoning capabilities. These blocks are chosen for their straightforward visual representation and their utility in demonstrating fundamental patterns like distributed channel building.
- **API examples:** high-level API blocks that illustrate how to implement common aggregate computing patterns directly within ScaFi, such as the construction of communication channels, without the need for composing lower-level blocks.
- **Actuations:** blocks that enable interaction with the simulation environment, such as moving nodes or changing their colour, to visualize algorithm outcomes and dynamics.
- **Sensing:** blocks designed to gather information from the environment or the node itself, including sensing boolean conditions or retrieving node identifiers, essential for context-aware computing.

Movement API: the movement API within ScaFi-Blocks provides a high-level abstraction for controlling node mobility in the simulated environment. This API allows users to specify movement strategies and behaviours, such as flocking or pattern formation, directly through visual blocks. This approach simplifies the integration of dynamic behaviours into aggregate algorithms, making the exploration of mobile sensor networks and robotic swarms more accessible.

Simulated Environment: ScaFi-Blocks is complemented by a simulated environment that acts as a virtual testbed for swarm algorithms. This environment supports the execution of ScaFi-Blocks programs, allowing users to observe the emergent behaviours of their algorithms in real time. It is equipped with features to simulate various conditions, including node movement and local actuation, offering a rich platform for experimentation and learning.

Code Generation: a key feature of ScaFi-Blocks is its ability to generate executable ScaFi code from the visual blocks. This code-generation process not only bridges the gap between visual programming and textual code but also serves as an educational tool, enabling users to learn ScaFi syntax and concepts through direct comparison. The generated code can be exported and run within the ScaFi ecosystem, facilitating a smooth transition from visual design to practical implementation.

Blockly Workspace Session: at the core of the ScaFi-Blocks interface is the Blockly workspace, a flexible and interactive area where users can assemble their algorithms. The workspace session is designed to be intuitive, supporting drag-and-drop composition, real-time feedback, and session saving/loading functionalities. This ensures a user-friendly experience, encouraging experimentation and iterative design processes. The Blockly workspace is instrumental in making aggregate computing concepts tangible, allowing users to visualize and manipulate the building blocks of their algorithms cohesively and engagingly.

4 ScaFi-Blocks Design and Implementation

4.1 Design Principles

The ScaFi-Blocks environment is crafted around several core principles aimed at enhancing *usability*, *learning*, and *engagement*:

User-Friendly Interface: at its heart, ScaFi-Blocks is engineered to be intuitive, employing a drag-and-drop interface for algorithm design. This method significantly lowers the cognitive load and potential for errors, making the system accessible to non-experts and encouraging collaborative algorithm development.

Rapid Feedback Loop: recognizing the importance of immediate feedback in swarm programming, ScaFi-Blocks should ensure users receive instant execution results in a simulated environment. This feature, inspired by environments such as Scratch and Snap, is crucial for understanding the effects of algorithmic modifications, thus accelerating the learning process for the ScaFi framework and its paradigms.

Knowledge Transfer: to bridge the gap between visual and textual programming, ScaFi-Blocks automatically translates visual designs into ScaFi code. This functionality is pivotal for users to grasp underlying concepts and learn the ScaFi syntax through direct comparison, promoting a deeper understanding of algorithm structures.

Compositional Approach: reflecting the essence of aggregate computing, ScaFi-Blocks emphasizes the composition of complex algorithms from simple building blocks. This design principle showcases the framework’s power to create sophisticated behaviours from basic elements, aligning with the compositional nature of aggregate computing.

Minimalist Design: ScaFi-Blocks adopts a minimalist approach, providing a curated set of building blocks that simplify the design process while encapsulating the framework’s complexity. This strategy allows users to concentrate on algorithmic logic rather than the intricacies of the ScaFi framework or field calculus.

Engagement and Fun: Finally, ScaFi-Blocks is designed to be both enjoyable and educational, offering an interactive experience that makes learning the ScaFi framework an engaging adventure. Users can experiment within a simulated environment, discovering and visualizing fascinating behaviours and dynamics.

Together, these principles guide the development of ScaFi-Blocks, ensuring it serves as an accessible, educational, and enjoyable tool for both newcomers and experienced users in the field of swarm programming.

4.2 Architecture

The architecture of ScaFi-Blocks is delineated in Figure 2, illustrating its dual-component structure:

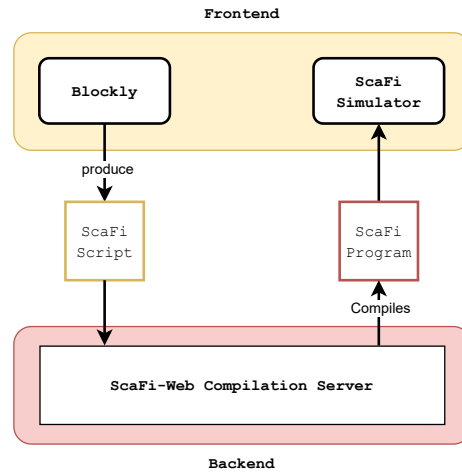


Fig. 2: The architecture of ScaFi-Blocks.

- **frontend:** this component constitutes the visual programming interface alongside the execution engine that runs compiled ScaFi programs;
- **backend:** it processes ScaFi-Web scripts, compiling them into a format executable by the frontend.

A typical workflow in ScaFi-Blocks involves the following steps:

1. the user designs an algorithm using the visual interface. Blockly then converts this visual representation into a textual format, adhering to predefined specifications (refer to the Blockly documentation² for further details).
2. This textual representation is forwarded to the backend, where it is compiled into an executable format for the frontend.
3. Finally, the compiled program is relayed back to the frontend for execution.

4.3 Implementation Details

ScaFi Blocks represents a significant enhancement to ScaFi Web by integrating the Blockly library as an alternative to the conventional textual editor. This integration facilitates a more interactive and visual programming experience. The setup process for a Blockly environment in a new scenario encompasses several key steps:

1. **Defining new blocks:** for each new block, it is essential to specify its characteristics, including the block's name, the number of inputs it requires, its colour, and the type of result it produces. These details are typically organized and stored in a JSON file, facilitating easy management and configuration.
2. **Describing the toolbox:** the toolbox is a critical component of the Blockly interface, acting as a categorization system for the blocks. It organizes the blocks into categories, making them accessible from the Blockly editor's menu (refer to Figure 1). This organization aids users in finding the appropriate blocks for their programming tasks.
3. **Defining generators:** generators are functions that, given a block, produce the corresponding string representation for that block, and determine the order of evaluation within the global expression. These generators convert Blockly's visual blocks into executable ScaFi code, suitable for running in the designated environment.

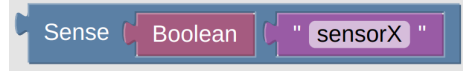
The implementation of block generators is predominantly achieved through JavaScript. However, to facilitate integration with ScaFi Web, the translation process is handled via Scala.js³ in this case. Specifically, a class is defined for each block type, encapsulating a strategy (i.e., a generator) to generate the ScaFi code corresponding to that block. Blocks are categorized into two main types:

- **Unit blocks:** these blocks do not yield a result and, as such, cannot be connected to other blocks. An example within our context is the output block, which acts as a terminal point for a block sequence.
- **Value blocks:** in contrast, value blocks produce a result and can be attached to other blocks as inputs or outputs. These blocks are fundamental for constructing meaningful sequences of operations.

² <https://developers.google.com/blockly/guides/create-custom-blocks/blockly-developer-tools>

³ <https://github.com/cric96/ScaFi-Blocks>

To illustrate, consider the implementation of the *sense* block, which is designed to interact with sensors:



The Scala.js code snippet below outlines the class definition for this block type:

```
class SenseBlockType extends ValueBlockType:
  override def name: String = "sense"
  override def order: Order = Orders.ORDER_ATOMIC
  override def fieldNames: Seq[String] = Seq()
  override def inputNames: Seq[String] = Seq("TYPE", "SENSOR_NAME")
  override def generator: Generator = (block) =>
    val _type = Blockly.ScaFi.valueToCode(block, "TYPE", Orders.NONE)
    val sensorName = Blockly.ScaFi.valueToCode(block, "SENSOR_NAME", Orders.NONE)
    val code = s"sense[${_type}] (${sensorName})"
    (code, Orders.NONE)
```

In this code snippet, the generator function is particularly noteworthy. It utilizes the `Blockly.ScaFi.valueToCode` method to extract and transform the values associated with the `"TYPE"` and `"SENSOR_NAME"` inputs into the appropriate ScaFi code snippet. Consequently, when the *sense* block is used within the Blockly editor, it results in the generation of ScaFi code resembling:

```
sense[TYPE] ("SENSOR_NAME")
```

This approach, whereby each block type is associated with a specific class that defines its behaviour and code generation logic, enables the seamless translation of visual programming constructs into executable ScaFi code.

5 Walkthrough

In this section, we discuss three comprehensive examples that can be implemented with the system we developed that demonstrate all the functionalities of the developed system, which are the *gradient*, the *channel*, and *flocking*. These examples are shown in images Figure 3, Figure 5, and Figure 6, each with the following two parts:

- the first line shows the implementation through Blockly;
- the last line shows a run of the simulation at three different time instants.

Gradient example: One of the most idiomatic blocks in aggregate programming, it allows computing the distance from areas classified as sources. It is a foundational block because, on top of it, other spatio-temporal constructs such as the channel can be built. The gradient, in ScaFi, is part of a library block described in the following manner:

```
def gradient(source: Boolean, metric: Metric): Double
```

Thus, to call it, it is typically described in the following way:

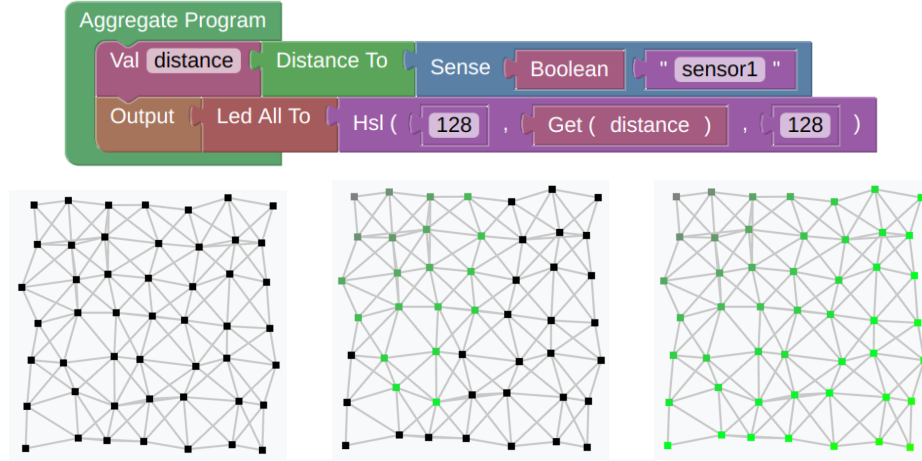


Fig. 3: The distance block example.

```
gradient(sense("source"), nbrRange)
```

Here, we wanted to hide the complexity of passing a metric to calculate this data structure, so we hid this step and allowed the developer to define only the source. Perception was also captured with a block (“sense” in the example) to which the name of the sensor and the desired type are passed. Furthermore, here we have made the system colour itself using the output as a saturation value and using the actuations exposed by ScaFi web. To do this and make the code more readable, we relied on a variable that was then used in evaluating the HSL value.

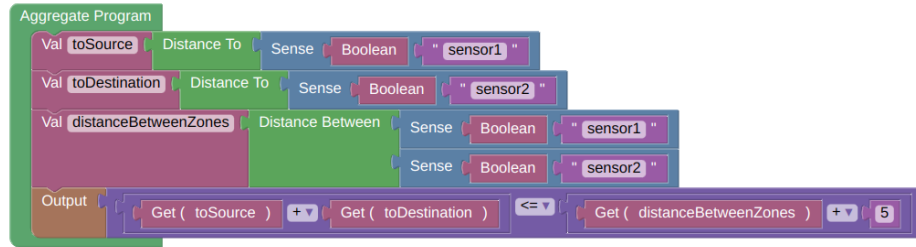


Fig. 4: The channel block built with ScaFi-Blocks.

Channel example: this block produces a spatial structure that connects an area marked as a source and an area marked as a destination. This structure is capable of *self-heal*, so in cases of changes in topology, the system finds another stable

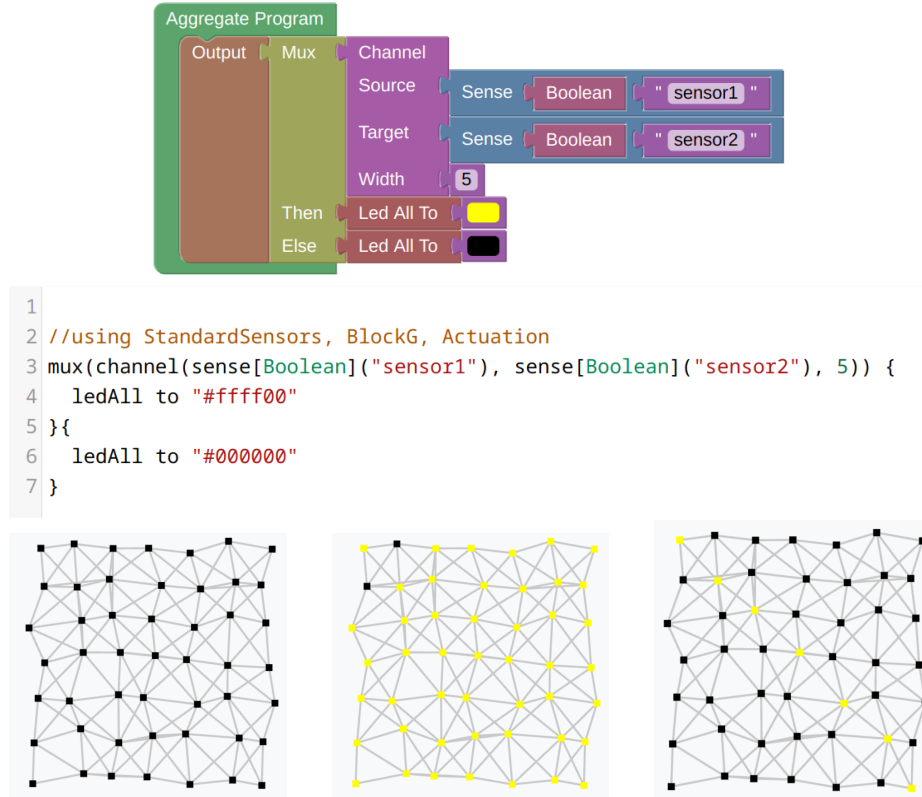


Fig. 5: The channel example, in the second line the code shown in the “show code” block.

configuration where the channel reforms following the shortest path between these zones. The channel can be recreated directly through the blocks exposed by our system (see, for example, Figure 4). But in this case, we have also made available the API block through the ScaFi system. Moreover, ScaFi Blocks offers the possibility to see the generated code using “show block” (second line in the reference figure). Additionally, to colour the gradient, we have conditions to colour the nodes based on the channel’s value. To do this, we used the `mux` block which accepts three blocks, the condition, the result when the condition is true, and the result when the condition is false.

Flocking example: To also show the system’s ability to perform actuations, we integrated a small example in which we show how it is possible to move nodes using flocking policies as described by Reynolds [46]. This part of the library derives from ScaFi web and allows moving the nodes according to a velocity vector. In this case, the library is much more complex and requires several parameters to be configured.

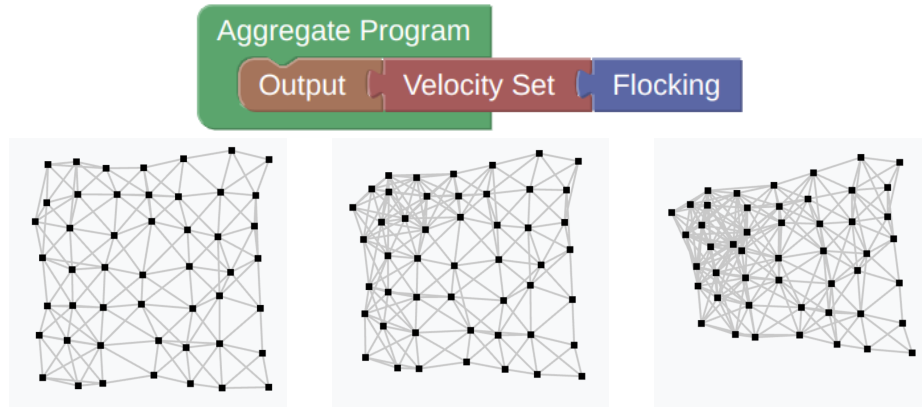


Fig. 6: A simple example of flocking implemented through ScaFi-Blocks.

Here is an example of minimalism: in this case, the flock has been summarized in a single block that actually translates into the following code:

```
//using StandardSensors, AdvancedFlock, BlockG, FlockLib, Movement2D, BlockT, CustomSpawn,
// Actuation, BlocksWithGC, BlockS
velocity set FlockBehaviour(attractionForce = 0.001, alignmentForce = 0.1,
    repulsionForce = 0.5, separationDistance = 10.0,
).run()
```

All the variables in this case distracted the developer against the intent, which was to describe a flock of nodes. Finally, to effectively perform the movement, the program must return the `velocity set` block, which, in the ScaFi web, is a block that sets the velocity of the nodes and is used to move the nodes in the simulation environment.

6 Related Work

There exist some recent reviews on visual programming, most notably on its application for end-user development [33], on the foundations of low-code [9], and on the comparison of visual and textual approaches for education [41]. There are also reviews/applications on the use of visual/low-code environments for distributed computing, e.g., on orchestration of distributed computations in the Internet of Things (IoT) [48], on the development of multiagent systems [34], on low-code IoT engineering [29], and on the human-computer interaction with social robots [21].

In the literature, the majority of works for visual and end-user robot programming focus on individual robots [4,21,52]. A few visual approaches, however, have also been proposed specifically for multi-robot systems and swarms. One approach is *FaMe (BPMN-driven FrAmework for Multi-robot systEms development)* [22], which adopts the *Business Process Modelling Notation (BPMN)* standard, and specifically its collaboration diagrams, and targets the ROS 2

platform. FaMe supports reusable behaviour and distributed execution. In this work, we focus primarily on collectives, whereas FaMe targets more heterogeneous composites [37]. The *Multi-Agent script Programming Language for multipotent Ensembles (Maple-Swarm)* [31] is an approach that extends the concept of hierarchical task networks for defining partial plans, also leveraging the abstraction of agent groups. Maple-Swarm provides a graphical frontend designer for visual programming, which however somewhat mixes code within the blocks corresponding to swarm tasks, and hence cannot be regarded as a low-code tool.

Other proposals in the literature are less related and provide, e.g., some visual or low-code support for the design of single robots or ad-hoc platforms, or visual languages lacking any low-code orientation. For instance, in [13], visual DSLs are defined to promote low-code configuration of ROS2 setups and integration of ROS with message brokers at the edge or cloud. In *SeSAm* [30], a visual programming environment is combined with agent-based simulation, but the visual language follows a quite “high-code”, imperative style not promoting compositionality of behaviours and usability.

7 Conclusions

In this paper, we have presented ScaFi-Blocks, a novel low-code swarm programming environment that leverages the visual programming paradigm to democratize the development of swarm algorithms. By building upon the ScaFi aggregate computing framework, ScaFi-Blocks offers a user-friendly interface that simplifies the design and implementation of distributed algorithms for swarm systems. The environment lowers the entry barrier for non-expert users and facilitates a broader engagement with swarm programming.

Through a detailed discussion of the design principles, architecture, and features of ScaFi-Blocks, we have demonstrated its potential to foster innovation and collaboration in the field of swarm programming. The examples of gradient, channel, and flocking algorithms illustrated in this paper underscore the practicality and versatility of ScaFi-Blocks in enabling the design of sophisticated swarm behaviours with minimal coding effort. This preliminary work has outlined the foundation and capabilities of ScaFi-Blocks, a novel low-code environment for swarm programming. Looking ahead, our future research will focus on a comprehensive evaluation of ScaFi-Blocks through user studies and case studies. Specifically, we aim to identify the target user group for this tool, examining the prerequisite knowledge required for its effective use. Additionally, while the design principles of ScaFi-Blocks have been methodologically followed, comparative evaluations with similar approaches remain unaddressed. We plan to expand our evaluation to include aspects such as usability, learning curves, and user engagement, enhancing the overall effectiveness of the environment. Moreover, we also plan to extend the programming environment with additional features and building blocks, e.g., by leveraging the MacroSwarm library [3], further enhancing its capabilities and usability.

Acknowledgments. This work was supported by the Italian PRIN project “CommonWears” (2020 HCWWLP) and the EU/MUR FSE PON-R&I 2014-2020.

References

1. Abd Alrahman, Y., De Nicola, R., Loret, M.: Programming interactions in collective adaptive systems by relying on attribute-based communication. *Science of Computer Programming* **192** (2020). <https://doi.org/10.1016/j.scico.2020.102428>
2. Aguzzi, G., Casadei, R., Maltoni, N., Pianini, D., Viroli, M.: ScaFi-Web: A web-based application for field-based coordination programming. In: *Coordination Models and Languages - 23rd IFIP WG 6.1 International Conference, COORDINATION 2021, Held as Part of the 16th International Federated Conference on Distributed Computing Techniques, DisCoTec 2021, Proceedings. Lecture Notes in Computer Science*, vol. 12717, pp. 285–299. Springer (2021). https://doi.org/10.1007/978-3-030-78142-2_18
3. Aguzzi, G., Casadei, R., Viroli, M.: MacroSwarm: A field-based compositional framework for swarm programming. In: *Coordination Models and Languages - 25th IFIP WG 6.1 International Conference, COORDINATION 2023, Held as Part of the 18th International Federated Conference on Distributed Computing Techniques, DisCoTec 2023, Proceedings. Lecture Notes in Computer Science*, vol. 13908, pp. 31–51. Springer (2023). https://doi.org/10.1007/978-3-031-35361-1_2
4. Ajaykumar, G., Steele, M., Huang, C.: A survey on end-user robot programming. *ACM Comput. Surv.* **54**(8), 164:1–164:36 (2022). <https://doi.org/10.1145/3466819>
5. Armoni, M., Meerbaum-Salant, O., Ben-Ari, M.: From scratch to "real" programming. *ACM Trans. Comput. Educ.* **14**(4), 25:1–25:15 (2014). <https://doi.org/10.1145/2677087>
6. Audrito, G., Casadei, R., Damiani, F., Salvaneschi, G., Viroli, M.: Functional programming for distributed systems with XC. In: *36th European Conference on Object-Oriented Programming, ECOOP 2022. LIPIcs*, vol. 222, pp. 20:1–20:28. Schloss Dagstuhl - Leibniz-Zentrum für Informatik (2022). <https://doi.org/10.4230/LIPIcs.ECOOP.2022.20>
7. Audrito, G., Casadei, R., Damiani, F., Viroli, M.: Computation Against a Neighbour: Addressing Large-Scale Distribution and Adaptivity with Functional Programming and Scala. *Logical Methods in Computer Science* **Volume 19, Issue 1** (Jan 2023), <https://lmcs.episciences.org/10826>
8. Beal, J., Pianini, D., Viroli, M.: Aggregate programming for the internet of things. *Computer* **48**(9), 22–30 (2015). <https://doi.org/10.1109/MC.2015.261>, <https://doi.org/10.1109/MC.2015.261>
9. Bock, A.C., Frank, U.: In search of the essence of low-code: An exploratory study of seven development platforms. In: *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion, MODELS 2021 Companion*, pp. 57–66. IEEE (2021). <https://doi.org/10.1109/MODELS-C53483.2021.00016>
10. Boissier, O., Bordini, R.H., Hubner, J., Ricci, A.: *Multi-agent oriented programming: programming multi-agent systems using JaCaMo*. Mit Press (2020)
11. Bonabeau, E., Dorigo, M., Théraulaz, G.: *Swarm intelligence: from natural to artificial systems*. Santa Fe Institute Studies in the Sciences of Complexity, Oxford university press (1999)
12. Brambilla, M., Ferrante, E., Birattari, M., Dorigo, M.: Swarm robotics: a review from the swarm engineering perspective. *Swarm Intell.* **7**(1), 1–41 (2013), <https://doi.org/10.1007/s11721-012-0075-2>

13. Brouzos, R., Panayiotou, K., Tsardoulas, E.G., Symeonidis, A.L.: A low-code approach for connected robots. *J. Intell. Robotic Syst.* **108**(2), 28 (2023). <https://doi.org/10.1007/S10846-023-01861-Y>
14. Busoniu, L., Babuska, R., Schutter, B.D.: A comprehensive survey of multiagent reinforcement learning. *IEEE Trans. Syst. Man Cybern. Part C* **38**(2), 156–172 (2008), <https://doi.org/10.1109/TSMCC.2007.913919>
15. Carroll, M., Namjoshi, K.S., Segall, I.: The Resh programming language for multirobot orchestration. In: *IEEE International Conference on Robotics and Automation, ICRA 2021*. pp. 4026–4032. IEEE (2021), <https://doi.org/10.1109/ICRA48506.2021.9561133>
16. Casadei, R.: Artificial collective intelligence engineering: A survey of concepts and perspectives. *Artif. Life* **29**(4), 433–467 (2023). https://doi.org/10.1162/ARTL_A_00408
17. Casadei, R.: Macroprogramming: Concepts, state of the art, and opportunities of macroscopic behaviour modelling. *ACM Comput. Surv.* **55**(13s), 275:1–275:37 (2023). <https://doi.org/10.1145/3579353>
18. Casadei, R., Dente, F., Aguzzi, G., Pianini, D., Viroli, M.: Self-organisation programming: A functional reactive macro approach. In: *IEEE International Conference on Autonomic Computing and Self-Organizing Systems, ACSOS 2023*. pp. 87–96. IEEE (2023). <https://doi.org/10.1109/ACSOS58161.2023.00026>
19. Casadei, R., Fortino, G., Pianini, D., Placuzzi, A., Savaglio, C., Viroli, M.: A methodology and simulation-based toolchain for estimating deployment performance of smart collective services at the edge. *IEEE Internet Things J.* **9**(20), 20136–20148 (2022). <https://doi.org/10.1109/JIOT.2022.3172470>
20. Casadei, R., Viroli, M., Aguzzi, G., Pianini, D.: ScaFi: A Scala DSL and toolkit for aggregate programming. *SoftwareX* **20**, 101248 (2022), <https://doi.org/10.1016/j.softx.2022.101248>
21. Coronado, E., Mastrogiovanni, F., Indurkha, B., Venture, G.: Visual programming environments for end-user development of intelligent and social robots, a systematic review. *J. Comput. Lang.* **58**, 100970 (2020). <https://doi.org/10.1016/J.COLA.2020.100970>
22. Corradini, F., Pettinari, S., Re, B., Rossi, L., Tiezzi, F.: A bpmn-driven framework for multi-robot system development. *Robotics Auton. Syst.* **160**, 104322 (2023). <https://doi.org/10.1016/J.ROBOT.2022.104322>
23. De Nicola, R., Loreti, M., Pugliese, R., Tiezzi, F.: A formal approach to autonomic systems programming: The SCEL language. *ACM Trans. Auton. Adapt. Syst.* **9**(2), 7:1–7:29 (2014). <https://doi.org/10.1145/2619998>
24. Dedousis, D., Kalogeraki, V.: A framework for programming a swarm of UAVs. In: *11th Pervasive Technologies Related to Assistive Environments Conference (PETRA'18)*, Proceedings. pp. 5–12. ACM (2018). <https://doi.org/10.1145/3197768.3197772>
25. Fernandez-Marquez, J.L., Serugendo, G.D.M., Montagna, S., Viroli, M., Arcos, J.L.: Description and composition of bio-inspired design patterns: a complete overview. *Nat. Comput.* **12**(1), 43–67 (2013). <https://doi.org/10.1007/S11047-012-9324-Y>
26. Francesca, G., Birattari, M.: Automatic design of robot swarms: Achievements and challenges. *Frontiers Robotics AI* **3**, 29 (2016). <https://doi.org/10.3389/FROBT.2016.00029>, <https://doi.org/10.3389/frobt.2016.00029>
27. Francesca, G., Brambilla, M., Brutschy, A., Trianni, V., Birattari, M.: Automode: A novel approach to the automatic design of control software for

- robot swarms. *Swarm Intell.* **8**(2), 89–112 (2014). <https://doi.org/10.1007/S11721-014-0092-4>, <https://doi.org/10.1007/s11721-014-0092-4>
28. Hüttenrauch, M., Sosc, A., Neumann, G.: Deep reinforcement learning for swarm systems. *J. Mach. Learn. Res.* **20**, 54:1–54:31 (2019), <http://jmlr.org/papers/v20/18-476.html>
 29. Ithirwe, F., Ruscio, D.D., Mazzini, S., Pierini, P., Pierantonio, A.: Low-code engineering for internet of things: a state of research. In: *MODELS '20: ACM/IEEE 23rd International Conference on Model Driven Engineering Languages and Systems, Companion Proceedings*. pp. 74:1–74:8. ACM (2020). <https://doi.org/10.1145/3417990.3420208>
 30. Klügl, F.: Sesam: Visual programming and participatory simulation for agent-based models. In: *Multi-Agent Systems - Simulation and Applications*, pp. 477–507. Computational Analysis, Synthesis, and Design of Dynamic Systems, CRC Press / Taylor & Francis (2009). <https://doi.org/10.1201/9781420070248.CH16>
 31. Kosak, O., Huhn, L., Bohn, F., Wanninger, C., Hoffmann, A., Reif, W.: Mapleswarm: Programming collective behavior for ensembles by extending HTN-planning. In: *Leveraging Applications of Formal Methods, Verification and Validation: Engineering Principles - 9th International Symposium on Leveraging Applications of Formal Methods, ISoLA 2020, Proceedings, Part II. Lecture Notes in Computer Science*, vol. 12477, pp. 507–524. Springer (2020). https://doi.org/10.1007/978-3-030-61470-6_30
 32. Koutsoubelias, M., Lalis, S.: TeCoLa: A programming framework for dynamic and heterogeneous robotic teams. In: *Proceedings of the 13th International Conference on Mobile and Ubiquitous Systems: Computing, Networking and Services (MobiQuitous 2016)*. pp. 115–124. ACM (2016). <https://doi.org/10.1145/2994374.2994397>
 33. Kuhail, M.A., Farooq, S., Hammad, R., Bahja, M.: Characterizing visual programming approaches for end-user developers: A systematic review. *IEEE Access* **9**, 14181–14202 (2021). <https://doi.org/10.1109/ACCESS.2021.3051043>
 34. Lemée, J., Burattini, S., Mayer, S., Ciortea, A.: Domain-expert configuration of hypermedia multi-agent systems in industrial use cases. In: Agmon, N., An, B., Ricci, A., Yeoh, W. (eds.) *Proceedings of the 2023 International Conference on Autonomous Agents and Multiagent Systems, AAMAS 2023, London, United Kingdom, 29 May 2023 - 2 June 2023*. pp. 2499–2501. ACM (2023). <https://doi.org/10.5555/3545946.3598981>, <https://dl.acm.org/doi/10.5555/3545946.3598981>
 35. Lima, K., Marques, E.R.B., Pinto, J., Sousa, J.B.: Dolphin: A task orchestration language for autonomous vehicle networks. In: *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS 2018*. pp. 603–610. IEEE (2018). <https://doi.org/10.1109/IROS.2018.8594059>
 36. Mamei, M., Zambonelli, F., Leonardi, L.: Co-fields: A physically inspired approach to motion coordination. *IEEE Pervasive Comput.* **3**(2), 52–61 (2004). <https://doi.org/10.1109/MPRV.2004.1316820>
 37. Masolo, C., Vieu, L., Ferrario, R., Borgo, S., Porello, D.: Pluralities, collectives, and composites. In: *Formal Ontology in Information Systems - Proceedings of the 11th International Conference, FOIS 2020. Frontiers in Artificial Intelligence and Applications*, vol. 330, pp. 186–200. IOS Press (2020). <https://doi.org/10.3233/FAIA200671>
 38. Mone, G.: Rise of the swarm. *Commun. ACM* **56**(3), 16–17 (2013). <https://doi.org/10.1145/2428556.2428562>, <https://doi.org/10.1145/2428556.2428562>

39. Myers, B.A.: Visual programming, programming by example, and program visualization: a taxonomy. In: Proceedings of the SIGCHI Conference on Human Factors in Computing Systems, CHI 1886. pp. 59–66. ACM (1986). <https://doi.org/10.1145/22627.22349>
40. Ni, Y., Kremer, U., Stere, A., Iftode, L.: Programming ad-hoc networks of mobile and resource-constrained devices. In: Proceedings of the 2005 ACM SIGPLAN conference on Programming language design and implementation. PLDI05, ACM (Jun 2005). <https://doi.org/10.1145/1065010.1065040>
41. Noone, M., Mooney, A.: Visual and textual programming languages: a systematic review of the literature. *Journal of Computers in Education* **5**(2), 149–174 (Mar 2018). <https://doi.org/10.1007/s40692-018-0101-5>
42. Oh, H., Shirazi, A.R., Sun, C., Jin, Y.: Bio-inspired self-organising multi-robot pattern formation: A review. *Robotics Auton. Syst.* **91**, 83–100 (2017). <https://doi.org/10.1016/J.ROBOT.2016.12.006>
43. Pinciroli, C., Beltrame, G.: Swarm-oriented programming of distributed robot networks. *Computer* **49**(12), 32–41 (2016). <https://doi.org/10.1109/MC.2016.376>, <https://doi.org/10.1109/MC.2016.376>
44. Pinho, D., Aguiar, A., Amaral, V.: What about the usability in low-code platforms? A systematic literature review. *J. Comput. Lang.* **74**, 101185 (2023). <https://doi.org/10.1016/J.COLA.2022.101185>
45. Ray, P.P.: A survey on visual programming languages in internet of things. *Sci. Program.* **2017**, 1231430:1–1231430:6 (2017). <https://doi.org/10.1155/2017/1231430>, <https://doi.org/10.1155/2017/1231430>
46. Reynolds, C.W.: Flocks, herds and schools: A distributed behavioral model. In: Stone, M.C. (ed.) Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques, SIGGRAPH 1987, Anaheim, California, USA, July 27–31, 1987. pp. 25–34. ACM (1987), <https://doi.org/10.1145/37401.37406>
47. Rokis, K., Kirikova, M.: Challenges of low-code/no-code software development: A literature review. In: Perspectives in Business Informatics Research - 21st International Conference on Business Informatics Research, BIR 2022, Proceedings. Lecture Notes in Business Information Processing, vol. 462, pp. 3–17. Springer (2022). https://doi.org/10.1007/978-3-031-16947-2_1
48. Silva, M., Dias, J.P., Restivo, A., Ferreira, H.S.: A review on visual programming for distributed computation in iot. In: Computational Science - ICCS 2021 - 21st International Conference, Proceedings, Part IV. Lecture Notes in Computer Science, vol. 12745, pp. 443–457. Springer (2021). https://doi.org/10.1007/978-3-030-77970-2_34
49. Trianni, V.: Evolutionary Swarm Robotics - Evolving Self-Organising Behaviours in Groups of Autonomous Robots, Studies in Computational Intelligence, vol. 108. Springer (2008), <https://doi.org/10.1007/978-3-540-77612-3>
50. Viroli, M., Audrito, G., Beal, J., Damiani, F., Pianini, D.: Engineering resilient collective adaptive systems by self-stabilisation. *ACM Trans. Model. Comput. Simul.* **28**(2), 16:1–16:28 (2018), <https://doi.org/10.1145/3177774>
51. Viroli, M., Beal, J., Damiani, F., Audrito, G., Casadei, R., Pianini, D.: From distributed coordination to field calculus and aggregate computing. *J. Log. Algebraic Methods Program.* **109** (2019). <https://doi.org/10.1016/j.jlamp.2019.100486>
52. Weintrop, D., Afzal, A., Salac, J., Francis, P., Li, B., Shepherd, D.C., Franklin, D.: Evaluating coblox: A comparative study of robotics programming environments for adult novices. In: Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems. CHI '18, ACM (2018). <https://doi.org/10.1145/3173574.3173940>